

UNIVERSIDADE VEIGA DE ALMEIDA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

GIOVANNI CARDOSO AVALLONE BELO
LUANA CRISTINA DE AZEVEDO CELESTINO
MARCOS FELIPE FERREIRA PIRES
PEDRO HENRIQUE GRACILIANO TAKA

PROCESSAMENTO E BUSCA DE TEXTOS:
IMPLEMENTAÇÃO DE UMA ESTRUTURA TRIE E DE UM MECANISMO DE
BUSCA COM ÍNDICE INVERTIDO, HASH E ANÁLISE DE COMPLEXIDADE

RIO DE JANEIRO

2026

GIOVANNI CARDOSO AVALLONE BELO
LUANA CRISTINA DE AZEVEDO CELESTINO
MARCOS FELIPE FERREIRA PIRES
PEDRO HENRIQUE GRACILIANO TAKA

PROCESSAMENTO E BUSCA DE TEXTOS:
IMPLEMENTAÇÃO DE UMA ESTRUTURA TRIE E DE UM MECANISMO DE
BUSCA COM ÍNDICE INVERTIDO, HASH E ANÁLISE DE COMPLEXIDADE

Trabalho apresentado à Universidade
Veiga de Almeida como requisito parcial
para a obtenção da nota da avaliação A1
na disciplina Análise e Otimização de
Sistemas, do curso de Bacharelado em
Sistemas de Informação.

Professor: André Lucio.
Turma: 4169CMPN8B_P1.

RIO DE JANEIRO
2026

RESUMO

A presente pesquisa é um estudo sobre estruturas de dados aplicadas ao processamento e à busca de textos, com foco na árvore de prefixos (Trie) e no índice invertido apoiado em tabela hash. A pesquisa foi aplicada por meio da implementação dessas estruturas em Python, sem bibliotecas prontas, sobre um corpus de 24 documentos técnicos em português que somam 98.718 palavras. Essa pesquisa tem como objetivo geral relacionar cada operação implementada à sua complexidade assintótica e confrontar a previsão teórica com a medição. Os métodos utilizados foram de natureza experimental, com abordagem quantitativa, apoiados em nove experimentos de medição e em 154 testes automatizados. De acordo com o estudo desenvolvido, a busca por prefixo na Trie mostrou-se independente do tamanho do vocabulário, a Trie comprimida reduziu em 55 % o número de nós e a consulta ao índice respondeu cerca de 1.900 vezes mais rápido que a varredura do texto com o algoritmo Knuth-Morris-Pratt. Por fim, a pesquisa constata que o custo de construir os índices, cerca de 330 ms, se paga em torno da sexta consulta.

Palavras-chave: Trie; Índice Invertido; Tabela Hash; Análise de Complexidade; Recuperação de Informação.

SUMÁRIO

1 INTRODUÇÃO	4
2 DESENVOLVIMENTO	5
2.1 VISÃO GERAL DA SOLUÇÃO	5
2.2 A TRIE E O AUTOCOMPLETE	6
2.3 PRÉ-PROCESSAMENTO, VOCABULÁRIO E ÍNDICE INVERTIDO	6
2.4 USO DE HASH.....	7
2.5 BUSCA POR SEQUÊNCIA COM KMP	8
2.6 ANÁLISE DE COMPLEXIDADE	8
2.7 EXEMPLOS DE CONSULTAS E RESULTADOS	9
2.8 QUESTÕES CONCEITUAIS DA PARTE I	10
2.9 LIMITAÇÕES DA SOLUÇÃO	11
3 CONSIDERAÇÕES FINAIS	12
REFERÊNCIAS.....	13

1 INTRODUÇÃO

Localizar uma informação em um grande volume de texto depende menos da velocidade com que o computador lê do que da estrutura de dados construída antes da primeira consulta. Esse é o princípio da área de Recuperação de Informação (MANNING; RAGHAVAN; SCHÜTZE, 2008) e o eixo deste trabalho, que trata de dois problemas complementares: o autocomplete, em que um prefixo digitado deve recuperar as palavras de um léxico que começam por ele, e a busca em documentos, em que um termo deve indicar os arquivos em que aparece.

Resolver qualquer um dos dois por varredura custa tempo proporcional ao tamanho da base a cada consulta. A alternativa estudada é pagar um custo maior uma única vez, na construção de índices, para que cada consulta seguinte seja barata. A árvore de prefixos, descrita por Fredkin (1960), atende ao primeiro problema; o índice invertido, ao segundo.

O objetivo geral é implementar essas estruturas sem bibliotecas prontas e relacionar cada operação à sua complexidade assintótica, verificada por medição. Como objetivos específicos, o trabalho se propõe a:

- a) implementar a Trie e aplicá-la a um sistema de autocomplete;
- b) reaproveitar a mesma Trie em um mecanismo de busca sobre uma pasta de arquivos de texto, com pré-processamento, vocabulário, índice invertido e hash;
- c) implementar, como extensão, a busca por sequência de caracteres com o algoritmo Knuth-Morris-Pratt;
- d) medir os custos previstos pela análise e discutir os limites da solução.

O código foi escrito em Python e verificado por 154 testes automatizados; uma interface web executa os mesmos algoritmos no navegador, para a demonstração. A seção 2 descreve a solução e a sua análise, e a seção 3 apresenta as considerações finais.

2 DESENVOLVIMENTO

Esta seção acompanha o caminho que os dados percorrem no sistema, do texto bruto à resposta de uma consulta, e associa a cada estrutura o seu custo.

2.1 VISÃO GERAL DA SOLUÇÃO

O sistema tem duas partes que compartilham a mesma Trie. Na Parte I, ela armazena um léxico de 8.641 palavras e responde ao autocomplete. Na Parte II, o vocabulário extraído dos documentos é inserido nessa mesma estrutura, e a busca por prefixo passa a ser a porta de entrada do índice invertido. O código é dividido em módulos, um por etapa (Trie, pré-processamento, radicalização, índice, KMP e integração), e a pasta de documentos é varrida a cada execução, sem nome de arquivo fixado no código: basta incluir um novo .txt para que ele seja indexado. As principais decisões de implementação foram usar a palavra sem acentos como caminho da Trie, com cada grafia como uma palavra; trocar a pontuação por espaço e descartar números; manter o índice pela palavra exata e pelo radical; e usar o *dict* do Python como tabela hash. A Tabela 1 reúne as estatísticas exigidas pelo enunciado.

Tabela 1 – Estatísticas obrigatórias medidas sobre o corpus

Métrica	Valor
Documentos processados	24
Palavras após a tokenização	98.718
Palavras após a remoção de stopwords	52.756
Termos distintos (vocabulário)	10.656
Palavras armazenadas na Trie	10.656
Radicais distintos no índice invertido	5.720
Tempo de leitura dos arquivos	4,8 ms
Tempo de pré-processamento	128,4 ms
Tempo de construção da Trie	43,0 ms
Tempo de construção do índice invertido	149,4 ms
Tempo total de construção	325,8 ms

Fonte: Elaborada pelos autores (2026), mediana de 15 execuções.

Nota: cada tempo é a mediana da sua fase; por isso o total, mediana das execuções completas, difere da soma das linhas.

Termos distintos e palavras na Trie dão o mesmo número, porque todo o vocabulário foi inserido: grafias como *análise* e *analise* dividem o caminho, mas são

duas palavras. O tempo de cada consulta, última estatística exigida, é exibido pelo programa ao lado do custo previsto pela análise e aparece na seção 2.7.

2.2 A TRIE E O AUTOCOMPLETE

Na Trie, cada aresta corresponde a um caractere, e o caminho da raiz até um nó representa um prefixo; um marcador de fim de palavra distingue as palavras armazenadas dos simples prefixos. Cada nó guarda um dicionário de filhos, esse marcador e as grafias originais da palavra. Como o acento distingue palavras, o caminho na árvore é a forma em minúsculas e sem acentos, mas cada grafia guardada no nó final é uma palavra: tanto *computa* quanto *computaç* encontram *computação*, que só existe com acento.

As três operações exigidas descem uma palavra de tamanho m em $O(m)$. A inserção percorre a chave criando os nós ausentes; a busca exata desce o caminho e verifica o marcador de fim de palavra; e a busca por prefixo desce o prefixo e coleta a subárvore alcançada por uma busca em profundidade com pilha explícita. Essa coleta custa $O(p)$, sendo p o número de nós da subárvore, e já produz a lista em ordem alfabética, porque os filhos são visitados em ordem de caractere. O custo total, $O(m + p)$, não depende do tamanho do léxico. O programa exibe todas as palavras encontradas, em páginas de 40.

Além do que o enunciado pede, a Trie recebeu duas extensões: sugestões pelas palavras mais frequentes, com busca *best-first* guiada pela maior frequência guardada em cada subárvore, e correção de digitação por distância de edição (LEVENSHTEIN, 1966; DAMERAU, 1964), calculada sobre a própria Trie, com poda dos ramos que já passaram do limite.

2.3 PRÉ-PROCESSAMENTO, VOCABULÁRIO E ÍNDICE INVERTIDO

O texto de cada arquivo passa por quatro etapas: conversão para minúsculas; troca da pontuação por espaço, para que *banco-de-dados* produza três palavras; tokenização por expressão regular, que extrai sequências de letras e descarta números; e remoção de *stopwords*, que elimina 46,6 % dos tokens. Como etapa opcional, foi implementado o radicalizador RSLP (ORENGO; HUYCK, 2001).

Radicalizar é reduzir as variantes de uma palavra a um radical comum só por regras de sufixo, sem dicionário, ao contrário da lematização. O RSLP aplica oito passos em ordem fixa (plural, feminino, advérbio, aumentativo, sufixo nominal, sufixo verbal, vogal final e acentos), cada um com regras de sufixo, substituição, tamanho mínimo do radical e exceções, das quais só a primeira que casa é aplicada: *algoritmos* perde o *s* no plural e o *o* na vogal final, e *computadores* chega a *comput* pelo sufixo nominal. É ele que permite à consulta *algoritmo* encontrar os arquivos que só escrevem *algoritmos*: pelo radical *algoritm* ela alcança 19 arquivos, dos quais 14 têm a forma exata, e o programa marca os outros 5.

O conjunto das palavras distintas forma o vocabulário, inserido na Trie da Parte I. O índice invertido associa cada termo aos documentos em que ele ocorre, e o nome vem da inversão do índice natural de uma coleção, o índice direto, que associa cada documento às suas palavras. A relação é a mesma, lida no sentido oposto, e transforma a pergunta sobre em quais arquivos um termo aparece de uma varredura de todos os arquivos em uma única consulta. A estrutura adotada tem o formato {termo: {documento: frequência}}, indexada pela palavra exata e pelo radical; a frequência permite ordenar os resultados pelo BM25 (ROBERTSON et al., 1994), que limita o peso de ocorrências repetidas e desconta documentos longos.

Na consulta por prefixo, a Trie devolve os termos e, para cada um, o índice da palavra exata informa os documentos em que ele aparece, listados abaixo do termo, como mostra a Figura 1. A consulta por palavra vai ao radical; com vários termos, cada um é uma consulta ao índice, e os documentos que contêm todos, obtidos por interseção a partir da menor lista, vêm antes dos que contêm só parte. Digitada sem acento, a palavra é outra grafia; o programa sugere a dos documentos.

2.4 USO DE HASH

O índice invertido opera sobre o *dict* do Python, uma tabela hash, como autoriza o enunciado. Uma função hash transforma a chave em uma posição de um vetor de tamanho fixo, de modo que localizar um termo não exige compará-lo com os demais: o custo médio da busca é $O(1)$. Para tornar esse mecanismo visível e mensurável, foi implementada também uma tabela própria, com hash polinomial calculada pelo esquema de Horner.

Como há infinitas chaves possíveis e finitas posições, duas chaves podem cair na mesma posição, o que caracteriza a colisão. Ela foi tratada por encadeamento: cada posição guarda uma lista, percorrida na busca, cujo comprimento médio é o fator de carga, a razão entre elementos e posições. Com os 5.720 radicais do índice, a lista média das posições ocupadas caiu de 11,17 elementos, com fator de carga 11,17, para 1,39, com fator 0,70. Por isso a tabela dobra de capacidade quando o fator passa de 0,75, operação $O(n)$ cujo custo amortizado por inserção continua constante (CORMEN et al., 2012). No pior caso, com todas as chaves na mesma posição, a busca degeneraria para $O(n)$.

2.5 BUSCA POR SEQUÊNCIA COM KMP

Como extensão opcional, uma terceira modalidade procura uma sequência de caracteres diretamente no texto original, sem índice, com o algoritmo de Knuth, Morris e Pratt (1977). Ao falhar depois de casar j caracteres, a busca ingênua recomeça uma posição adiante; o KMP aproveita que esses caracteres já são conhecidos e consulta uma função de falha, calculada em $O(m)$ a partir do padrão, que indica o maior prefixo próprio que também é sufixo do trecho casado. O ponteiro do texto nunca retrocede e o número de passos não excede $2n$, o que garante $O(n + m)$ para qualquer entrada, contra $O(n \cdot m)$ da busca ingênua no pior caso.

2.6 ANÁLISE DE COMPLEXIDADE

O Quadro 1 resume o custo das principais operações implementadas.

Quadro 1 – Complexidade das principais operações

Operação	Custo
Trie: inserir e buscar palavra	$O(m)$
Trie: buscar por prefixo	$O(m + p)$
Tabela hash: inserir e buscar	$O(1)$ em média; $O(n)$ no pior caso
Índice invertido: consulta com q termos	$O(q)$ em média, mais $O(d \log d)$ para ordenar
KMP: função de falha e busca	$O(m)$ e $O(n)$
Busca ingênua de sequência	$O(n \cdot m)$ no pior caso
Construção completa do sistema	$O(N + V \cdot m)$

Fonte: Elaborada pelos autores (2026).

Nota: m = tamanho da palavra ou do padrão; n = tamanho do texto; p = nós abaixo do prefixo; d = documentos devolvidos; N = tokens do corpus; V = tamanho do vocabulário.

A Tabela 2 confronta essas previsões com os experimentos do projeto. A mais importante é a da primeira linha: quando o vocabulário dobra e o número de resultados é mantido, o tempo da Trie fica estável enquanto o da lista dobra, confirmando $O(m + p)$ contra $O(V \cdot m)$.

Tabela 2 – Previsões da análise confrontadas com as medições

Experimento	Medida	Resultado
Vocabulário dobra de 2.664 para 5.328 palavras, com 23 resultados	Tempo da Trie	27,9 μ s $\rightarrow$ 28,3 μ s
O mesmo experimento, na lista	Tempo da lista	21,8 ms $\rightarrow$ 45,4 ms
Vocabulário completo, de 10.656 palavras, com 73 resultados	Tempo: Trie $\times$ lista	78,8 μ s $\times$ 69,2 ms
Trie tradicional $\times$ Trie comprimida	Nós alocados	31.175 $\times$ 14.040
KMP $\times$ busca ingênua, pior caso ($n = 16.000$; $m = 60$)	Comparações	31.941 $\times$ 956.460
KMP $\times$ busca ingênua, texto natural (5 padrões)	Comparações	razão de 1,00 a 1,02

Fonte: Elaborada pelos autores (2026), a partir de benchmark.py.

Nota: cada medida soma as consultas a 8 prefixos, com até 10 resultados por prefixo.

No pior caso construído, o KMP efetua cerca de $2n$ comparações, 30 vezes menos que a busca ingênua; em texto natural, porém, os dois empatam, porque as falhas ocorrem logo nos primeiros caracteres. A vantagem do KMP é a garantia do limite linear, e não a velocidade média.

2.7 EXEMPLOS DE CONSULTAS E RESULTADOS

A Tabela 3 compara o tempo das modalidades de consulta, e a Figura 1 reproduz consultas feitas nas duas partes do programa.

Tabela 3 – Tempo das modalidades de consulta

Consulta	Estrutura utilizada	Custo	Tempo
algoritmo (palavra)	índice invertido (hash)	$O(1)$	31,4 μ s
rede neural (dois termos)	índice invertido	$O(q)$	33,9 μ s
comp (prefixo, 151 termos)	Trie e índice invertido	$O(m + p)$	479,9 μ s
chave pública (sequência)	KMP sobre o texto	$O(N)$	58,5 ms

Fonte: Elaborada pelos autores (2026), mediana de 301 execuções (21 para a sequência).

A consulta ao índice é cerca de 1.900 vezes mais rápida que a varredura com KMP, e essa razão cresce com a coleção, porque um lado é constante e o outro é linear. Os 325,8 ms de construção se pagam, assim, em torno da sexta consulta. A

consulta por prefixo é a mais cara entre as indexadas porque, além de percorrer a subárvore, acessa o índice uma vez para cada um dos 151 termos.

Figura 1 – Consultas realizadas nas duas partes do programa

```
[Parte I] Digite o prefixo: prog
Palavras encontradas:
  programa
  programação
  programações
  programadas
  ...

[Parte II] Digite o prefixo: compil
Palavras encontradas:
  compila                ->  2 documento(s)
    compiladores.txt, linguagens_programacao.txt
  ...
  compilações            ->  1 documento(s)
    compiladores.txt
  ...
-- 10 de 13 termos. Enter mostra mais; 0 encerra a lista:

[Parte II] Digite a palavra: rede neural
Termos da consulta:
  rede      radical 'red'  -> 17 arquivo(s)
  neural    radical 'neur' ->  7 arquivo(s)
Arquivos com todos os termos: 7 (com algum deles: 17)
- aprendizagem_profunda.txt    353 ocorrência(s)  BM25 3,829  [2/2 termos]
- aprendizado_de_maquina.txt   26 ocorrência(s)  BM25 3,415  [2/2 termos]
...

[Parte II] Digite a palavra: algortimo
'algortimo' não foi encontrada em nenhum documento.
...
Você quis dizer: algoritmo?
```

Fonte: Saída do programa (2026).

2.8 QUESTÕES CONCEITUAIS DA PARTE I

1. Por que uma Trie é adequada para sistemas de autocomplete? Porque o prefixo digitado é um caminho na árvore: localizá-lo custa $O(m)$, qualquer que seja o tamanho do léxico, e todas as palavras que começam por ele estão na subárvore alcançada, e em nenhum outro lugar. Cada nova letra digitada exige descer apenas mais um nível.

2. Qual a vantagem de uma Trie em relação à busca sequencial em uma lista de palavras? A lista precisa testar todas as V palavras, a um custo $O(V \cdot m)$, enquanto a Trie descarta ramos inteiros no primeiro caractere divergente e custa $O(m + p)$. No vocabulário de 10.656 palavras, a Trie foi cerca de 880 vezes mais rápida (Tabela 2), e a diferença cresce com o léxico.

3. Como palavras que possuem prefixos comuns são representadas na Trie? Elas compartilham fisicamente o caminho até o ponto em que divergem, de modo que o prefixo comum é armazenado uma única vez: *computação*, *computacional* e *computador* dividem os nós de *computa* e só então se separam. Um mesmo nó pode encerrar uma palavra e ser caminho de outras.

4. Qual a complexidade de busca de uma palavra de tamanho m ? $O(m)$: são m passos, um por caractere, cada um com custo constante no dicionário de filhos, independentemente do número de palavras armazenadas. A busca pode terminar antes, no primeiro caractere inexistente.

5. Que problema de memória pode ocorrer em uma Trie tradicional? Cada caractere ocupa um nó com o seu próprio dicionário de filhos, mesmo em trechos sem bifurcação. No corpus, 10.656 palavras ocuparam 31.175 nós, cerca de 2,9 por palavra, boa parte em cadeias de filho único. O problema se agrava com alfabetos grandes e chaves longas e pouco compartilhadas.

6. O que é uma Trie comprimida e como ela procura reduzir esse problema? É a variante em que cada cadeia de nós de filho único vira um único nó, rotulado com a substring correspondente (MORRISON, 1968). Ela reconhece as mesmas palavras, com a mesma complexidade, mas precisa dividir arestas na inserção. No corpus, reduziu os nós de 31.175 para 14.040, uma economia de 55,0 %.

2.9 LIMITAÇÕES DA SOLUÇÃO

A solução tem limites que convém registrar. Todas as estruturas ficam em memória e são reconstruídas a cada execução, o que é irrelevante para 24 documentos, mas inviável para milhões, caso em que o índice precisaria ser gravado em disco e atualizado de forma incremental. O RSLP é baseado em regras e agressivo: *computação* e *computador* geram radicais diferentes e não se encontram na mesma consulta, problema que um lematizador resolveria. O índice guarda frequências, e não posições, e por isso não responde a buscas por frase. Por fim, a busca por sequência varre o corpus inteiro a cada consulta; estruturas como o vetor de sufixos responderiam em tempo proporcional ao padrão, ao custo de uma construção mais cara.

3 CONSIDERAÇÕES FINAIS

O trabalho implementou, sem bibliotecas prontas, uma Trie aplicada ao autocomplete e um mecanismo de busca que reaproveita essa estrutura, integrando pré-processamento, vocabulário, índice invertido e tabela hash, além da busca por sequência com KMP. Os objetivos propostos foram cumpridos, e as medições confirmaram as previsões da análise assintótica: a busca por prefixo não dependeu do tamanho do vocabulário, a tabela hash reproduziu a relação entre fator de carga e comprimento das listas, e o KMP respeitou o limite linear no pior caso.

O resultado de síntese é o contraste entre consultar um índice e varrer o texto: cerca de 31 microssegundos contra 58 milissegundos para responder em quais arquivos um termo aparece. Esse contraste justifica a existência dos índices, cujo custo de construção se paga em poucas consultas. As extensões, sugestões por frequência, correção de digitação e consultas com vários termos, mostraram que a mesma Trie comporta novos usos sem ser refeita, e as limitações apontadas indicam os próximos passos: persistência do índice, busca por frase e estruturas de sufixo para a busca de sequências.

REFERÊNCIAS

CORMEN, T. H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

DAMERAU, F. J. A technique for computer detection and correction of spelling errors. **Communications of the ACM**, v. 7, n. 3, p. 171-176, 1964.

FREDKIN, E. Trie memory. **Communications of the ACM**, v. 3, n. 9, p. 490-499, 1960.

KNUTH, D. E.; MORRIS, J. H.; PRATT, V. R. Fast pattern matching in strings. **SIAM Journal on Computing**, v. 6, n. 2, p. 323-350, 1977.

LEVENSHTEIN, V. I. Binary codes capable of correcting deletions, insertions, and reversals. **Soviet Physics Doklady**, v. 10, n. 8, p. 707-710, 1966.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to information retrieval**. Cambridge: Cambridge University Press, 2008.

MORRISON, D. R. PATRICIA: practical algorithm to retrieve information coded in alphanumeric. **Journal of the ACM**, v. 15, n. 4, p. 514-534, 1968.

ORENGO, V. M.; HUYCK, C. A stemming algorithm for the Portuguese language. In: INTERNATIONAL SYMPOSIUM ON STRING PROCESSING AND INFORMATION RETRIEVAL, 8., 2001, Laguna de San Rafael. **Proceedings [...]**. Los Alamitos: IEEE Computer Society, 2001. p. 186-193.

ROBERTSON, S. E. et al. Okapi at TREC-3. In: TEXT RETRIEVAL CONFERENCE, 3., 1994, Gaithersburg. **Proceedings [...]**. Gaithersburg: NIST, 1994. p. 109-126.